NMK GLOBAL INC.

NMK Certification Portal

Document Title	NMK Certification Portal
Author	V. Prathyusha
Maintainer	V. Prathyusha
Developers	Sushmitha, Sai Kiran, Sohail, Gopi Kiran
Date	March 6, 2026

Distribution List

Recipient	Role	Responsibilities
V. Prathyusha	Author / Maintainer	Overall documentation, architecture design, system oversight
Sushmitha	Developer	Admin Dashboard (Create/Assign Exams), Candidate Results analytics
Sai Kiran	Developer	Exam creation, LLM API integration, question generation pipeline
Sohail	Developer	Login, registration, user profile, AWS Cognito authentication flows
Gopi Kiran	Developer	Candidate exam interface, timer, answer saving, results display

TABLE OF CONTENTS

1. Executive Summary .....
- 1.1 Business Value Delivered .....
- 1.2 System at a Glance .....

2. Project Scope & Objectives .....
- 2.1 In Scope .....
- 2.2 Out of Scope .....

3. Stakeholders & User Roles .....
- 3.1 Primary Users .....
- 3.2 Role Permissions Matrix .....

4. Technology Stack .....

5. System Architecture .....
- 5.1 High-Level Architecture Diagram .....
- 5.2 Authentication & Token Flow .....
- 5.3 Complete Data Flow Reference .....

6. Project & File Structure .....

7. Database Design .....
- 7.1 Entity Relationship Summary .....
- 7.2 Model Field Reference .....
- 7.3 Enum Types & Status Lifecycles .....
- 7.4 Indexing & Performance Notes .....

8. Backend API Reference .....
- 8.1 Auth — /auth .....
- 8.2 Admin — Exam Management .....
- 8.3 Candidate — Exam Taking .....

9. Frontend Pages Reference .....
- 9.1 Candidate Screens .....
- 9.2 Admin Screens .....
- 9.3 Shared Utility Functions .....

10. Application Screenshots .....
- 10.1 Login Page .....
- 10.2 Admin — Create Exam .....
- 10.3 Admin — Manage Exams .....
- 10.4 Admin — Assign Exams .....
- 10.5 Candidate — Available Exams .....
- 10.6 Exam Interface — Question View .....
- 10.7 Results Page .....
- 10.8 Admin — All Candidate Results .....

11. AWS Services Configuration .....
11.1 AWS Cognito .....
11.2 Gmail SMTP .....

12. Security Design .....
12.1 Authentication & Token Security .....
12.2 Authorisation & Role Enforcement .....
12.3 Data Access Controls .....
12.4 Input Validation .....

13. Environment Configuration .....

14. Installation & Local Setup .....

15. Key User Workflows .....
15.1 Admin: Full Exam Lifecycle .....
15.2 Candidate: First-Time Onboarding .....
15.3 Session Resume (Browser Refresh) .....

16. Complete API Flow Reference (Frontend → Backend → DB) .....
16.1 Authentication Flows .....
16.2 Admin — Exam Creation .....
16.3 Admin — Candidate Assignment .....
16.4 Candidate — Exam Taking .....

17. Email Notification System .....

18. Analytics & Reporting .....

19. Known Issues & Recommendations .....

20. Glossary .....

1. Executive Summary

The NMK Certification Portal is a full-stack, cloud-native examination management system designed and developed for NMK Global Inc. It provides a complete, end-to-end certification lifecycle — from exam creation and candidate assignment through timed exam delivery, real-time answer persistence, automated scoring, and results analytics — within a single, secure, role-separated platform.

The system eliminates manual exam coordination by centralising every certification touchpoint into one integrated platform. Administrators can create exams with AI-generated questions on demand, assign them to candidates with automated onboarding, and monitor performance through a live analytics dashboard. Candidates receive a professional portal to take exams with a live countdown timer, auto-save progress, and immediate results with detailed question-by-question review.

1.1 Business Value Delivered

Objective	How This System Achieves It
Eliminate manual exam paper creation	AI-powered LLM API generates topic-specific MCQs on demand — no human authoring required
Centralise certification operations	Single platform covers every stage: create → assign → take → score → analyse
Enforce secure access control	AWS Cognito + JWT RS256 + role-based route guards ensure strict data separation
Automated candidate onboarding	New candidates auto-provisioned in Cognito + DB with Gmail SMTP credential delivery
Prevent exam progress loss	Real-time answer auto-save + server-side resume endpoint — no lost progress on refresh
Provide candidate performance visibility	Admin results dashboard with filter-by-status, filter-by-exam, and average score metrics
Scale with organisation	Cloud-native AWS Cognito; PostgreSQL for structured data; stateless FastAPI backend

1.2 System at a Glance

Dimension	Detail
Platform Type	Full-stack web application (internal, SaaS-style)
User Roles	Candidate (exam taker) · Admin (NMK staff — @nmkglobalinc.com)
Authentication	AWS Cognito (User Pool, JWT tokens, email verification)
Frontend	Python + Streamlit (session-based auth, multi-screen UI)
Backend	Python + FastAPI (RESTful, 16 endpoints, OpenAPI documented)
Database	PostgreSQL via SQLAlchemy ORM (UUID primary keys, 5 tables)
Question AI	External LLM API (HTTP REST, batch generation, retry logic)

Dimension	Detail
Email	Gmail SMTP via smtplib (STARTTLS port 587, automated notifications)
Hosting (dev)	localhost:8501 (frontend) · localhost:8000 (backend)
API Docs	http://localhost:8000/docs (Swagger UI auto-generated)

2. Project Scope & Objectives

2.1 In Scope

- Admin account login and session management via AWS Cognito with JWT RS256 tokens
- Candidate account creation, login, email verification, and password change via AWS Cognito
- AI-powered exam creation: admin specifies title, language, question count, and time limit; LLM generates MCQs
- Exam management: activate, deactivate, view question count and metadata per exam
- Candidate assignment: assign exams to email lists with automatic Cognito provisioning and Gmail SMTP notification
- Timed exam delivery with client-side JS countdown timer and server-side elapsed time tracking
- Paginated exam interface (10 questions per page) with radio-button answer selection
- Real-time answer auto-save on page navigation via bulk-save endpoint
- Full exam session resume after browser close or refresh via GET /exam/resume
- Automated scoring on submission: correct answers compared server-side, percentage computed
- Detailed results page: per-question correct/wrong review with correct answer revealed
- Admin analytics: total attempts, completed count, in-progress count, average score, per-candidate breakdown
- Change password functionality via Cognito admin API with old password verification

2.2 Out of Scope

- Video or audio proctoring during exams
- Manual question authoring (all questions generated by LLM API)
- Multi-language or i18n support in the UI
- Mobile-native application (web-only via Streamlit browser UI)
- Payment or subscription management
- Third-party calendar integrations (Google Calendar, Outlook)

3. Stakeholders & User Roles

3.1 Primary Users

Role	Who They Are	Primary Goals	System Access Level
Candidate	Employees or external users assigned certification exams	Take assigned exams, review results, change password	Own exam attempts only — cannot see other candidates
Admin	NMK Global hiring/training staff (@nmkglobalinc.com)	Create exams via LLM, assign to candidates, monitor results	All exams, all candidate attempts, full analytics

3.2 Role Permissions Matrix

Action	Candidate	Admin
View assigned exams	▯ Own assigned only	▯ All exams
Start / take exam	▯ Assigned exams only	▯ N/A
Save answers & resume	▯ Own exam only	▯ N/A
Submit exam & view results	▯ Own exam only	▯ N/A
Create exams via LLM API	▯ No	▯ Yes
Activate / deactivate exams	▯ No	▯ Yes (own exams only)
Assign exams to candidates	▯ No	▯ Yes
Auto-provision new Cognito users	▯ No	▯ Yes (via assign endpoint)
View all candidate results	▯ No	▯ Yes — all attempts
Filter and export results data	▯ No	▯ Yes — by status and exam
Change own password	▯ Yes	▯ Yes

4. Technology Stack

Layer	Technology	Version	Role in System
Frontend	Python · Streamlit	1.32+	Multi-screen web UI, session management, exam interface, admin dashboard
Backend API	Python · FastAPI	0.110+	RESTful API, dependency injection, 16 endpoints, OpenAPI/Swagger docs
ORM	SQLAlchemy	2.0+	Database models, session management, JSON column mutation tracking
DB Migrations	SQLAlchemy metadata	Auto	Tables auto-created via Base.metadata.create_all() on startup
Database	PostgreSQL	14+	Relational storage, UUID primary keys, 5 tables, FK constraints
Authentication	AWS Cognito	Managed	User Pool, JWT issuance, email verification, password policy
Token Decode	python-jose (RS256)	3.3+	Verify Cognito JWTs using JWKS public key endpoint (cached at startup)
AI / LLM	External LLM API	HTTP REST	On-demand MCQ generation by programming language and question count
Email	smtplib + Gmail SMTP	STARTTLS 587	Automated exam assignment emails with optional credentials for new users
Cloud Admin SDK	AWS boto3	Latest	Cognito admin operations: create user, set permanent password
Data Validation	Pydantic v2	2.0+	All request/response schema validation; EmailStr for candidate emails
Config	python-dotenv	1.0+	Load all secrets from .env file in both backend and frontend

5. System Architecture

5.1 High-Level Architecture Diagram

The platform follows a clean three-tier architecture. All inter-tier communication is authenticated and stateless.

```
┌──────────────────────────────────────────────────────────┐
│                  PRESENTATION LAYER                      │
│  Streamlit Frontend  (localhost:8501)                    │
│  login · exams · exam_ui · results · admin_dash          │
└──────────────────────────┬───────────────────────────────┘
                           │  HTTPS  Authorization: Bearer JWT
                           ▼
┌──────────────────────────────────────────────────────────┐
│                  BUSINESS LOGIC LAYER                    │
│  FastAPI Backend  (localhost:8000)                       │
│  /auth  /admin  /exams  /exam  /exam/resume              │
│  get_current_user() → JWT validation on every call       │
└──────────────────────────┬───────────────────────────────┘
                           │
   | SQLAlchemy ORM         | AWS + External Services
   ▼                        ▼
┌──────────────┐   ┌──────────────────────────────────┐
│ PostgreSQL   │   │ Cognito → User Pool + JWT        │
│ 5 Tables     │   │ LLM API → Question Generation    │
│ users        │   │ Gmail SMTP → Email Notify        │
│ exams        │   │ boto3 → Admin Cognito SDK        │
└──────────────┘   └──────────────────────────────────┘
```

5.2 Authentication & Token Flow

#	Actor	Action	System Response
1	**Candidate/Admin**	Enter email + password in Streamlit Login UI	Credentials passed to boto3 cognito.initiate_auth()
2	**AWS Cognito**	Validate credentials against User Pool	Returns IdToken (JWT RS256), AccessToken, RefreshToken
3	**Streamlit Frontend**	Store IdToken in st.session_state['access_token']	Token held in server-side memory only — never on disk
4	**Frontend**	Make API call with Authorization: Bearer <IdToken>	Every protected endpoint receives the token in header
5	**FastAPI**	get_current_user() extracts and validates JWT	Fetches JWKS public keys (cached at startup) → RS256 verify
6	**FastAPI**	Verify: signature, audience, issuer, expiry	Returns JWT payload (sub, email, exp) or raises HTTP 401
7	**FastAPI**	POST /auth/sync: upserts User in local DB	Extracts email; if @nmkglobalinc.com → is_admin=True
8	**Frontend**	Store is_admin flag in session state	Drives which pages are shown: Admin Dashboard vs Exams

5.3 Complete Data Flow Reference

All 16 API endpoints mapped: Frontend Action → Data Sent → Endpoint → DB/Cloud Operations → Response. See Section 15 for full detail.

6. Project & File Structure

```
certification portal/
├── backend/
│   └── app/
│       ├── __init__.py          ← Package init
│       ├── main.py              ← FastAPI app, all 16 routes, LLM integration
│       ├── db.py                ← SQLAlchemy engine, SessionLocal, get_db()
│       ├── models.py            ← All 5 ORM table definitions
│       ├── schemas.py           ← All Pydantic request/response schemas
│       ├── cognito_auth.py      ← JWT decode, JWKS cache, get_current_user()
│       ├── exam.py              ← compute_score() scoring engine
│       └── email_utils.py       ← Gmail SMTP, send_exam_assignment_email()
│
├── frontend/
│   └── app.py                   ← Streamlit UI (all screens in one file)
│       ├── init_session()        ← Session state initialisation
│       ├── login_page()          ← Cognito login, forgot password, register
│       ├── exam_list_page()      ← Available exams (candidate view)
│       ├── exam_ui()             ← Timed exam interface, paginated questions
│       ├── result_page()         ← Score display, question-by-question review
│       ├── admin_dashboard()     ← Tabs: Create/Manage/Assign/Results
│       └── auto_resume_exam_if_needed()  ← Session restore on login
│
├── create_db.py                 ← One-time DB init, admin seed, sample Qs
├── .env                         ← All secrets (NOT in version control)
└── requirements.txt             ← Python dependencies
```

7. Database Design

7.1 Entity Relationship Summary

Model / Table	Primary Key	Relationships	Key Purpose
users	String (Cognito sub)	Root identity record	Stores email, name, is_admin for all portal users
exams	UUID (gen_id)	created_by → users.id; has many questions + assignments	Exam records created by admin with LLM-generated questions
questions	UUID (gen_id)	exam_id → exams.id	Individual MCQs with choices JSON and answer_index
exam_assignments	UUID (gen_id)	exam_id → exams.id; assigned_by → users.id	Links an exam to a candidate email address; tracks status
candidate_exams	UUID (gen_id)	user_id → users.id; exam_id → exams.id	One record per candidate attempt; stores answers JSON + score

7.2 Model Field Reference

users

Column	Type	Constraint	Description
`id`	`String`	PK	Cognito sub UUID — set from JWT payload on first /auth/sync
`email`	`String`	Unique, Index	Unique email. Indexed for O(1) lookup on every API call.
`name`	`String`	Nullable	Display name — auto-set from email prefix on provisioning
`is_admin`	`Boolean`	Default: False	True for @nmkglobalinc.com users — grants admin dashboard access
`created_at`	`DateTime(TZ)`	Server default	Auto-set by DB server on INSERT

exams

Column	Type	Constraint	Description
`id`	`String`	PK, UUID	Auto-generated by gen_id() = str(uuid.uuid4())
`title`	`String`	NOT NULL	Exam display name entered by admin (e.g. 'sql', 'java')
`language`	`String`	NOT NULL	Programming language topic sent to LLM API
`question_count`	`Integer`	NOT NULL	Actual number of questions generated and stored
`time_allowed_secs`	`Integer`	NOT NULL	Time limit in seconds (UI input in minutes × 60)
`created_by`	`String`	FK → users.id	Cognito sub of the admin who created the exam
`created_at`	`DateTime(TZ)`	Server default	Auto-set creation timestamp
`is_active`	`Boolean`	Default: True	False hides exam from candidates; toggled via PATCH /toggle

questions

Column	Type	Constraint	Description
`id`	`String`	PK, UUID	Auto-generated by gen_id()
`text`	`String`	NOT NULL	Full question text as generated and parsed from LLM response
`choices`	`JSON`	NOT NULL	Python list as JSON: ['Option A', 'Option B', 'Option C', 'Option D']
`answer_index`	`Integer`	NOT NULL	Zero-based index of correct answer in choices array
`exam_id`	`String`	FK → exams.id	Parent exam (nullable for standalone seed questions)
`difficulty`	`Enum`	Default: easy	Difficulty enum: easy | medium | hard

exam_assignments

Column	Type	Constraint	Description
`id`	`String`	PK, UUID	Auto-generated by gen_id()
`exam_id`	`String`	FK → exams.id	Which exam is assigned
`candidate_email`	`String`	NOT NULL	Candidate email — may predate the user's DB account creation
`assigned_by`	`String`	FK → users.id	Admin who performed the assignment
`assigned_at`	`DateTime(TZ)`	Server default	Assignment timestamp
`status`	`String`	Default: 'assigned'	State: assigned → started → completed

candidate_exams

Column	Type	Constraint	Description
`id`	`String`	PK, UUID	Unique ID for this specific exam attempt
`user_id`	`String`	FK → users.id	The candidate taking the exam
`exam_id`	`String`	FK → exams.id	Which exam is being attempted
`question_ids`	`JSON`	Nullable	Ordered list of question UUIDs served to this candidate
`answers`	`JSON`	Nullable	{ 'question_id_str': selected_index_int } — updated on every save
`started_at`	`DateTime(TZ)`	Server default	Auto-set when attempt is first created
`ended_at`	`DateTime`	Nullable	Set to UTC now() on submission; null while in-progress
`status`	`String`	Default: 'not_started'	not_started → in_progress → completed / timed_out
`time_allowed_secs`	`Integer`	Default: 1800	Copied from exam at start time (30 min default)
`time_elapsed`	`Integer`	Default: 0	Updated on every answer save — used for accurate session resume
`score`	`Integer`	Default: 0	Final percentage (0–100) computed by compute_score() on submit

7.3 Enum Types & Status Lifecycles

Enum	Values
Difficulty (questions)	easy | medium | hard
ExamAssignment.status	assigned → started → completed
CandidateExam.status	not_started → in_progress → completed | timed_out

> *APPLICATION LIFECYCLE: assigned ──▶started (on POST /exam/{id}/start) ──▶completed (on POST /exam/{id}/submit)*
> *└──▶timed_out (when client timer reaches 0 and auto-submits)*

7.4 Indexing & Performance Notes

- email on users is Unique + Indexed — every API call resolves the current user by email. Critical for performance.
- candidate_email on exam_assignments is queried on every GET /exams call to filter assigned exams per candidate.
- user_id on candidate_exams is queried on save-answer, bulk-save, submit, and result — ensure this is indexed.
- answers column on candidate_exams is a mutable JSON dict — flag_modified() is required on every mutation to force SQLAlchemy UPDATE.
- Tables are auto-created by SQLAlchemy Base.metadata.create_all(). For production, migrate to Alembic for controlled schema migrations.

8. Backend API Reference

Base URL: http://localhost:8000 | Interactive Docs: http://localhost:8000/docs | All protected routes require: Authorization: Bearer {IdToken}

i *Token Type: The IdToken (not AccessToken) from Cognito is used as the Bearer token. It is decoded by cognito_auth.py using JWKS RS256 public keys cached at startup.*

8.1 Auth — /auth

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/auth/sync	Bearer	Header only	{message}	Upserts User from JWT. Sets is_admin=True for @nmkglobalinc.com. Call after every login.
GET	/auth/me	Bearer	None	{email, is_admin, name}	Returns current user record from local DB.
POST	/auth/change-password-admin	Bearer	{current_password, new_password}	{message}	Verifies old password via ADMIN_USER_PASSWORD_AUTH, then sets new permanent password.

8.2 Admin — Exam Management

Method	Endpoint	Auth	Request Body / Params	Returns	Notes

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
POST	/admin/exams	Admin Bearer	{title, language, question_count, time_allowed_secs}	ExamOut	LLM generation: batches of 10, up to 30 retries. Atomic DB commit of exam + all questions.
GET	/admin/exams	Admin Bearer	None	[ExamOut]	All exams ordered by created_at DESC.
PATCH	/admin/exams/{id}/toggle	Admin Bearer	Path: exam_id	{msg, is_active}	Flips is_active boolean. Active=False hides from candidates immediately.
POST	/admin/exams/{id}/assign	Admin Bearer	{candidate_emails: []}	{assigned_count, emailed_count, created_users}	Auto-provisions Cognito+DB users. Idempotent — skips duplicate email+exam pairs.
GET	/admin/exams/{id}/assignments	Admin Bearer	Path: exam_id	[{candidate_email, assigned_at}]	All candidates assigned to a specific exam.
GET	/admin/candidates/results	Admin Bearer	None	[{candidate, exam, status, score, timing}]	All CandidateExam records with user and exam details joined.

8.3 Candidate — Exam Taking

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/exams	Bearer	None	[ExamOut]	Active exams assigned to current candidate email. Filtered by ExamAssignment.
POST	/exam/{id}/start	Bearer	Path: exam_id	CandidateExamCreateOut	Creates CandidateExam (in_progress). Returns existing if already started (idempotent).

Method	Endpoint	Auth	Request Body / Params	Returns	Notes
GET	/exam/{id}	Bearer	Path: candidate_exam_id	{questions (no answer_index), timing}	Returns questions for exam interface. answer_index deliberately withheld.
POST	/exam/{id}/save-answer	Bearer	{question_id, selected_index, time_elapsed}	{msg}	Saves single answer + updates time_elapsed. Uses flag_modified() for JSON detection.
POST	/exam/{id}/bulk-save	Bearer	{answers: [{question_id, selected_index, time_elapsed}]}	{msg}	Batch save on page navigation. More efficient than one call per answer.
GET	/exam/resume	Bearer	None	{candidate_exam_id, questions, answers, timing}	Full exam state for auto-resume after browser close/refresh.
POST	/exam/{id}/submit	Bearer	{final_time_elapsed}	{score, status}	Finalise exam: status=completed, ended_at=now, compute_score() called.
GET	/exam/{id}/result	Bearer	Path: candidate_exam_id	{score, details[{question, choices, selected, correct_index, is_correct}]}	Full results. correct_index revealed only after submission.

9. Frontend Pages Reference

9.1 Candidate Screens

Screen / Function	Auth Required	Key Functionality
login_page()	No	Cognito login form, forgot password flow, new account registration, email verification. Redirect by role on success.
exam_list_page()	Candidate	Lists all active exams assigned to the logged-in candidate. Shows language, question count, time. Start Exam button.
exam_ui()	Candidate	Paginated exam interface (10 Qs/page). JS countdown timer. Radio button selection. Previous/Next page navigation with auto-save. Submit Exam button.
result_page()	Candidate	Score percentage with Pass/Fail indicator (threshold 70%). Question

Screen / Function	Auth Required	Key Functionality
		Review: ▯ correct (collapsed) / ▯ wrong (expanded with correct answer shown).
change_password_page()	Candidate	Two-field form (current + new password). Calls POST /auth/change-password-admin. Validates old password via Cognito before updating.

9.2 Admin Screens

Screen / Function	Auth Required	Key Functionality
admin_dashboard() — Create Exam tab	Admin	Form: title, language, question count, time limit. POST /admin/exams. Spinner during LLM call. Success/error confirmation.
admin_dashboard() — Manage Exams tab	Admin	Expandable exam cards: questions, time, status, UUID. GET /admin/exams. Deactivate/Activate toggle button per exam.
admin_dashboard() — Assign Exams tab	Admin	Dropdown of active exams + email textarea (one per line). POST /admin/exams/{id}/assign. Shows existing assignments below form.
admin_dashboard() — Candidate Results tab	Admin	Filter by status and exam. Metrics: total, completed, in-progress, average score. Scrollable results table with all attempt data.

9.3 Shared Utility Functions (frontend/app.py)

Function	Role
init_session()	Initialises all st.session_state keys on first load. Prevents KeyError on any page.
auto_resume_exam_if_needed()	Called immediately after login. Calls GET /exam/resume. If exam found, restores questions, answers, timer into session state silently.
api_get(endpoint) / api_post(endpoint, data)	Wrapper functions adding Authorization: Bearer header to all Streamlit→FastAPI calls.
save_dirty_answers()	Collects answers changed on the current page and calls POST /exam/{id}/bulk-save. Called on page nav and before submit.

10. Application Screenshots

10.1 Login Page

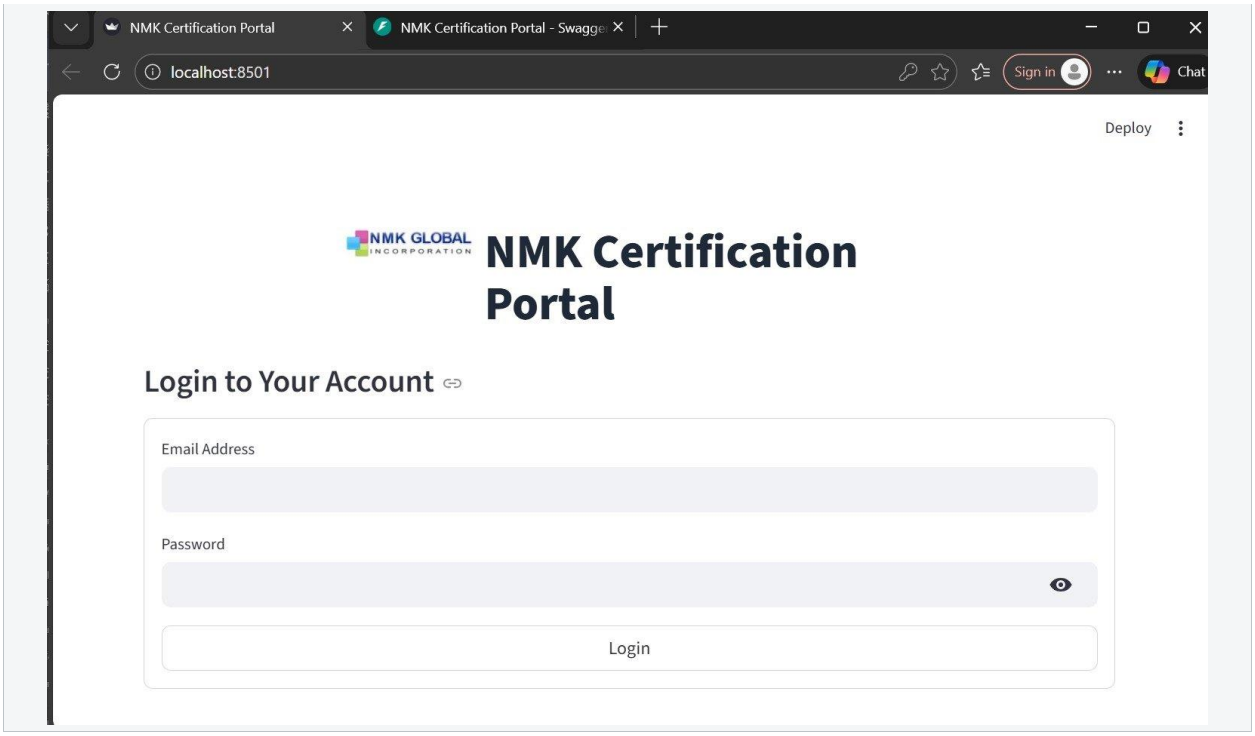

Figure: Login Page — NMK Certification Portal

UI Element	Description
NMK Global Logo	Brand header rendered on all pages. Loaded from frontend assets.
Email + Password	Credentials passed directly to AWS Cognito — never sent to FastAPI backend.
Login Button	Triggers initiate_auth(USER_PASSWORD_AUTH). On success: stores IdToken, calls /auth/sync, calls /auth/me.
Forgot Password	Calls Cognito forgot_password() → sends 6-digit code → confirm_forgot_password() to reset.

10.2 Admin — Create Exam

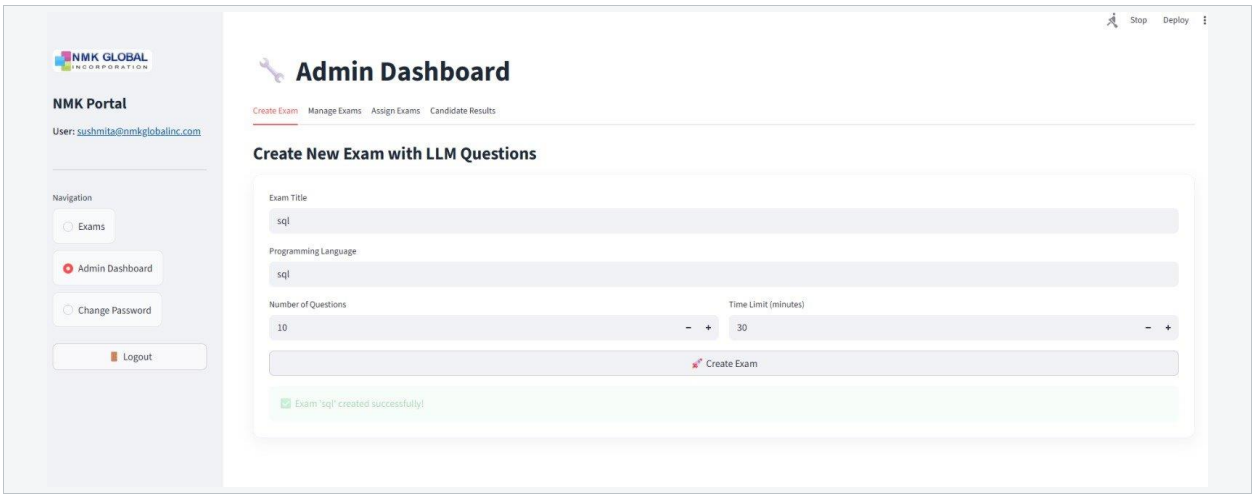

Figure: Admin Dashboard — Create Exam with LLM

Field	Description
Exam Title	Display name stored in exams.title (e.g. 'sql').
Programming Language	Sent to LLM API as question topic (e.g. 'sql', 'java', 'python').
Number of Questions	Requested count. Backend calls LLM in batches of 10 (max 30 retries).
Time Limit (minutes)	Multiplied × 60 and stored as time_allowed_secs in the exams table.
Create Exam	POST /admin/exams. Shows spinner. All questions committed atomically with the exam record.

10.3 Admin — Manage Exams

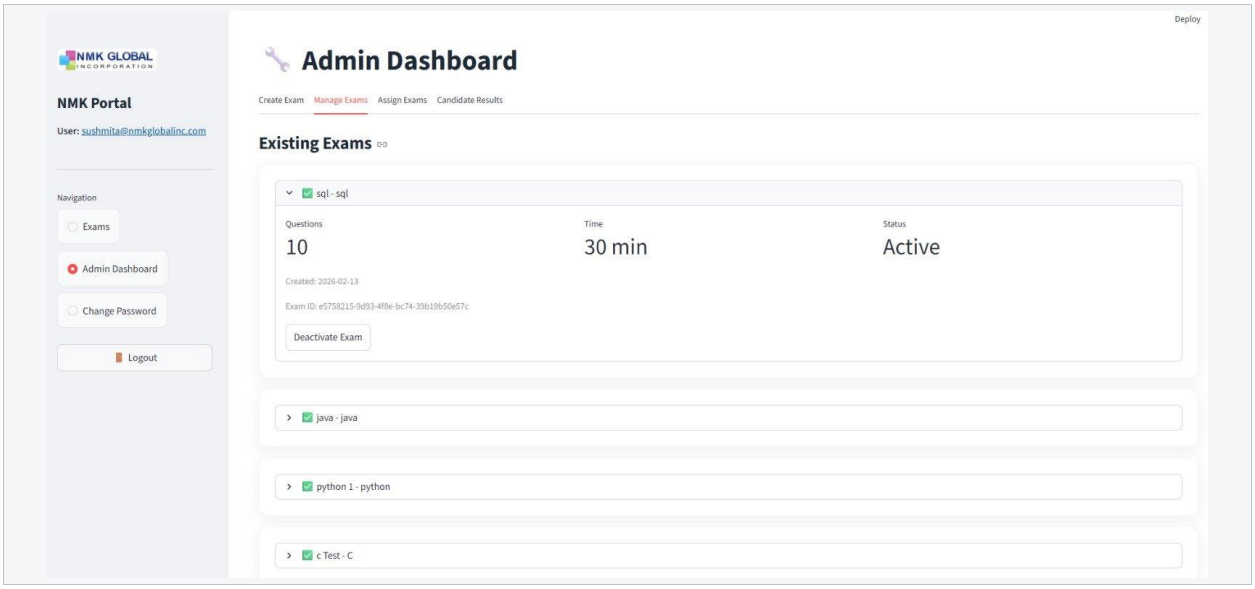

Figure: Admin Dashboard — Manage Exams

UI Element	Description
Exam expander	▸/▾ icon + title + language. Click to expand for details.
Questions / Time / Status metrics	From exams table: question_count, time_allowed_secs÷60, is_active.
Exam UUID	Primary key — useful for debugging API calls.
Deactivate/Activate button	PATCH /admin/exams/{id}/toggle. Flips is_active. Immediate effect on candidate exam list.

10.4 Admin — Assign Exams

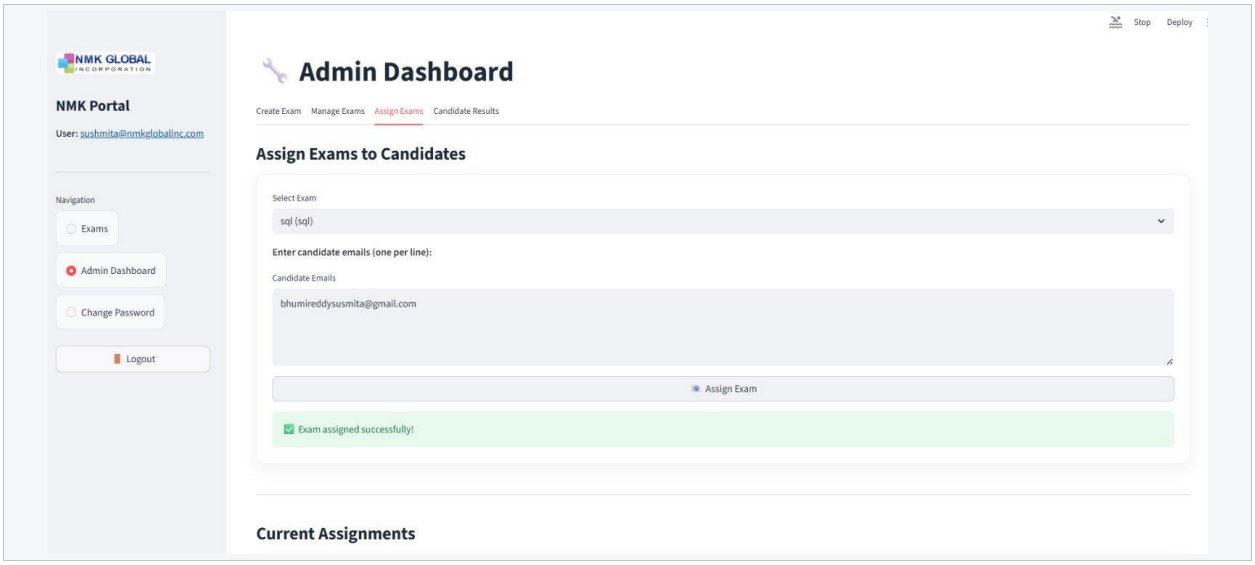

Figure: Admin Dashboard — Assign Exams to Candidates

UI Element	Description
Select Exam dropdown	Lists active exams from GET /admin/exams.
Candidate Emails textarea	One email per line. Each validated as EmailStr by Pydantic.
Assign Exam	POST /admin/exams/{id}/assign. Creates Cognito users, DB records, ExamAssignments. Sends emails.
Success message	Confirms all assignments created and emails sent.
Current Assignments table	Shows existing ExamAssignment records for the selected exam.

10.5 Candidate — Available Exams

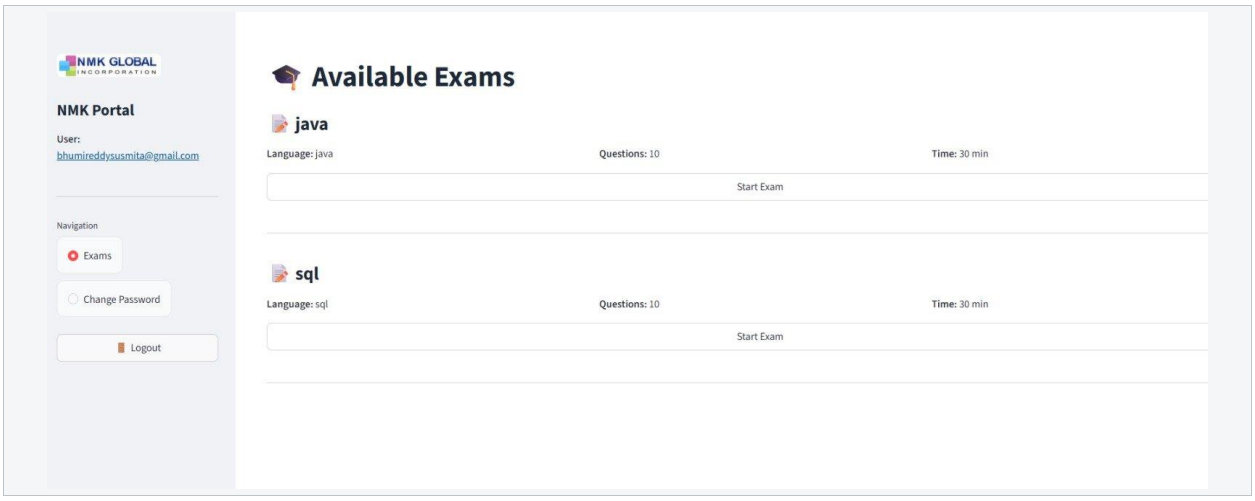

Figure: Candidate View — Available Exams

UI Element	Description
Available Exams header	Candidate-only view — admin users see Admin Dashboard instead.
Exam card (java / sql)	Title, language, question count, time limit from GET /exams filtered by candidate email and is_active=True.
Start Exam button	POST /exam/{exam_id}/start. Creates CandidateExam. Idempotent — returns existing if already started.

10.6 Exam Interface — Question View

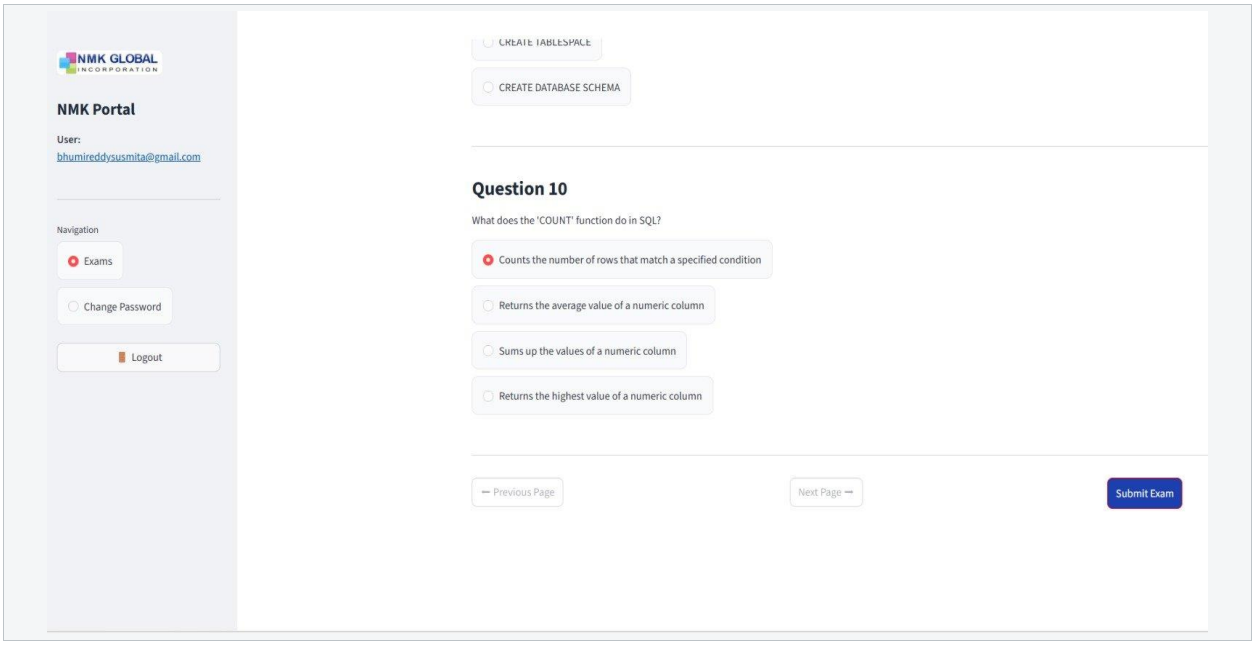

Figure: Candidate Exam Interface — Question View

UI Element	Description
Question text	LLM-generated question from questions.text column.
Radio buttons (4 choices)	Options from questions.choices JSON. answer_index never exposed to frontend.
Previous Page / Next Page	Calls save_dirty_answers() → POST /exam/{id}/bulk-save before changing page.
Submit Exam button	Saves remaining dirty answers then POST /exam/{id}/submit.
Countdown timer	JavaScript timer embedded via Streamlit. Re-seeded from server remaining time on each page load.

10.7 Results Page

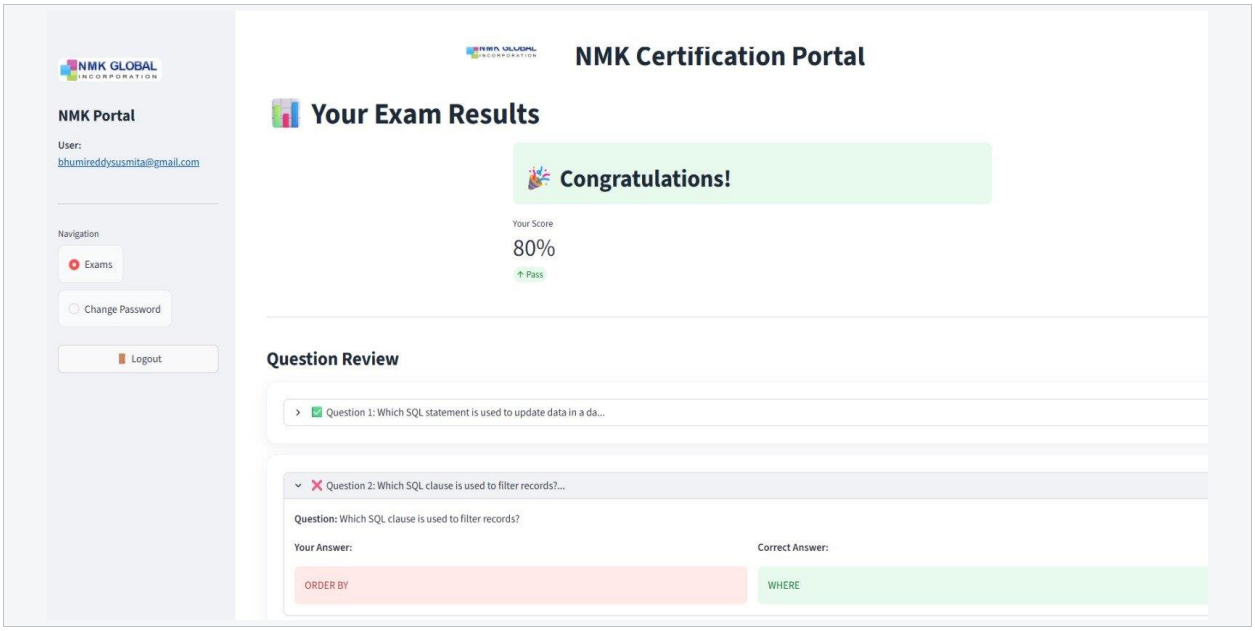

Figure: Candidate Results — Score & Question Review

UI Element	Description
Congratulations banner	Shown when score ≥ 70%. Green box. Warning for 50–69%. Error for < 50%.
Your Score: 80%	Integer from candidate_exams.score — computed by compute_score() on submission.
Pass / Fail indicator	≥70 = Pass (green ↑), 50–69 = borderline, <50 = Fail.
Question Review ▯	Collapsed by default for correct answers.
Question Review ▯	Expanded by default. Shows candidate's wrong answer (red) vs correct answer (green).

10.8 Admin — All Candidate Results

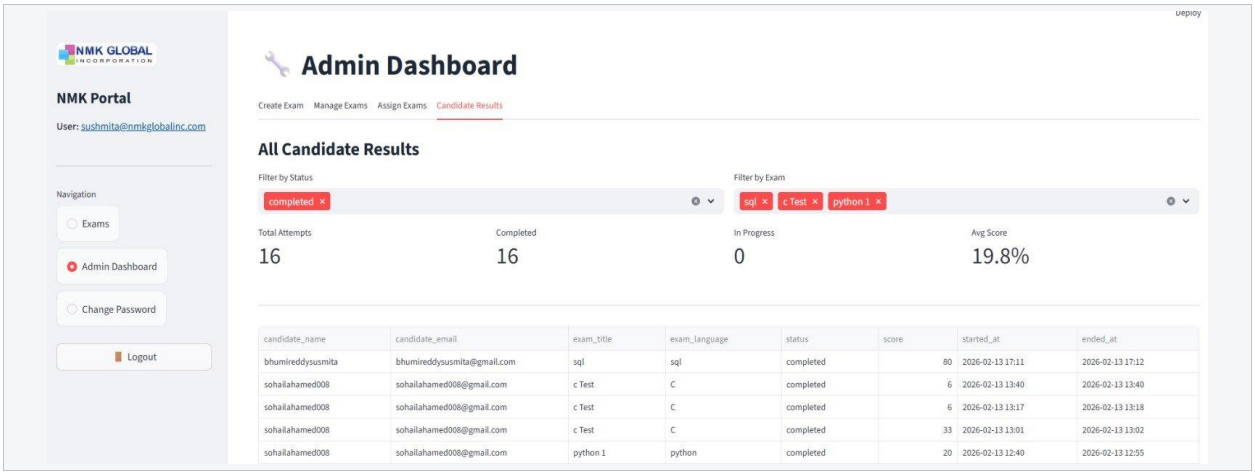

Figure: Admin Dashboard — All Candidate Results

UI Element	Description
Filter by Status	Multi-select: completed, in_progress, not_started, timed_out.
Filter by Exam	Multi-select: filter by exam title.
Total Attempts / Completed / In Progress	Counts computed from GET /admin/candidates/results response.
Avg Score: 19.8%	Mean of score column for completed attempts — computed in Streamlit via pandas.
Results table	Columns: candidate_name, candidate_email, exam_title, exam_language, status, score, started_at, ended_at.

11. AWS Services Configuration

11.1 AWS Cognito

Setting	Value / Detail
Service	AWS Cognito User Pool
Auth Flow	USER_PASSWORD_AUTH (must be enabled on App Client) + ADMIN_USER_PASSWORD_AUTH (for change password)
Token Type Used	IdToken — contains sub, email, exp, aud claims needed by backend
Email Verification	Required before first login. Cognito sends 6-digit code.
Password Reset	Cognito forgot_password() + confirm_forgot_password() flow
Admin Domain Rule	Users with @nmkglobalinc.com email automatically assigned is_admin=True in local DB on /auth/sync
JWKS Endpoint	https://cognito-idp.{region}.amazonaws.com/{pool_id}/.well-known/jwks.json
Token Signing	RS256 — asymmetric; public keys fetched and cached in cognito_auth.py at module load
Auto-Provisioning	Admin can call admin_create_user() + admin_set_user_password() via boto3 for new candidates

11.2 Gmail SMTP

Setting	Detail
Server	smtp.gmail.com (set via SMTP_SERVER env variable)
Port	587 (STARTTLS — upgrades plain TCP to TLS before credentials sent)
Auth	Gmail App Password (16-char code from Google Account Security → App Passwords)
Trigger: New Candidate	send_exam_assignment_email(send_password=True) — includes exam title + temporary password
Trigger: Existing Candidate	send_exam_assignment_email(send_password=False) — includes exam title + portal URL only

12. Security Design

12.1 Authentication & Token Security

- All protected API routes use FastAPI's HTTPBearer scheme — IdToken extracted from Authorization header on every request.
- Tokens are RS256-signed JWTs. JWKS public keys fetched from Cognito's well-known endpoint on module import and cached in memory.
- Expired tokens detected by python-jose — returns HTTP 401 'Invalid or expired token'. No stale auth possible.

- Invalid signatures or malformed tokens return HTTP 401. Internal error details are never exposed to the client.
- Streamlit stores IdToken in st.session_state (server-side memory only) — not in localStorage, reducing XSS exposure.

12.2 Authorisation & Role Enforcement

- Role enforcement occurs in the backend only — via the is_admin DB flag check on every admin endpoint.
- Admin routes explicitly check: if not db_user or not db_user.is_admin: raise HTTPException(403). No admin action possible without this check passing.
- Candidate exam access scoped by: CandidateExam.user_id == db_user.id on every exam endpoint — candidates cannot access other candidates' exams.

12.3 Data Access Controls

Resource	Access Rule
Candidate exam attempts	Owner candidate only (CandidateExam.user_id must match JWT sub)
Exam questions	No answer_index returned during exam — only revealed in result endpoint after submission
Admin exam list	Only users with is_admin=True in DB can call /admin/* endpoints
Candidate assignment	Only admins can assign exams; idempotent guard prevents duplicate assignments
All candidates results	Admin-only via GET /admin/candidates/results — HTTP 403 for non-admins

12.4 Input Validation

- All request bodies validated by Pydantic v2 schemas — malformed JSON or missing fields return HTTP 422 with detailed error messages.
- Candidate emails validated as EmailStr by Pydantic on every assignment request — prevents invalid formats reaching Cognito.
- SQL injection not possible via SQLAlchemy ORM — all queries use parameterised statements.
- email.strip().lower() normalisation prevents duplicate accounts from whitespace/case differences.

13. Environment Configuration

Security Notice: The .env file contains AWS credentials, database connection strings, and Cognito secrets. Add .env to .gitignore immediately. Use AWS Secrets Manager or equivalent in production.

13.1 Complete .env Reference

Variable	Used By	Required	Description

Variable	Used By	Required	Description
DATABASE_URL	backend/app/db.py	Yes	Full PostgreSQL connection: postgresql://user:pass@host:5432/dbname
AWS_REGION	Backend + Frontend	Yes	AWS region of Cognito User Pool (e.g. us-east-1)
COGNITO_USER_POOL_ID	cognito_auth.py, main.py	Yes	Cognito Pool ID format: us-east-1_XXXXXXXXX
COGNITO_CLIENT_ID	cognito_auth.py	Yes	App Client ID — must have USER_PASSWORD_AUTH + ADMIN_USER_PASSWORD_AUTH enabled
DEFAULT_PASSWORD	main.py (assign endpoint)	Yes	Temporary password for auto-provisioned candidates. Must meet Cognito password policy.
LLM_API_URL	main.py (create exam)	Yes	External LLM API endpoint. Accepts questionscount and language parameters.
SMTP_SERVER	email_utils.py	Yes	Gmail SMTP host: smtp.gmail.com
SMTP_PORT	email_utils.py	Yes	SMTP submission port: 587 (STARTTLS)
EMAIL_FROM	email_utils.py	Yes	Sender Gmail address (must match authenticated account)
EMAIL_PASSWORD	email_utils.py	Yes	Gmail App Password (16-character code — NOT Gmail login password)
PORTAL_URL	email_utils.py	Yes	Portal URL included in candidate assignment emails

14. Installation & Local Setup

14.1 Prerequisites

Prerequisite	Version	Purpose
Python	3.10+	Backend and frontend runtime
PostgreSQL	14+	Database server — must be running before backend starts
AWS Account	Active	Cognito User Pool with App Client configured
Gmail Account	Active	App Password enabled under Google Account Security
pip + venv	Latest	Python package manager and virtual environment isolation

14.2 Step 1 — Clone & Configure

Terminal

```
# Clone repository
git clone <repository-url>
cd certification_portal

# Create .env in project root (see Section 13 for all variables)
cp .env.example .env
```

```
# Edit .env with your actual values
```

14.3 Step 2 — Backend Setup

```
backend/
cd backend
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt

# Tables are auto-created by SQLAlchemy on first startup
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# ✓ API available at:     http://localhost:8000
# ✓ Swagger UI at:        http://localhost:8000/docs
```

14.4 Step 3 — Database Seed

```
create_db.py
# Run once from project root to create tables + seed admin + sample questions
python create_db.py

# Creates:
#   - All 5 tables via Base.metadata.create_all()
#   - Admin user: admin@nmk.com (id='admin-001', is_admin=True)
#   - 9 sample questions across easy/medium/hard difficulty levels
```

14.5 Step 4 — Frontend Setup

```
frontend/
cd frontend
python -m venv venv
source venv/bin/activate
pip install streamlit requests python-dotenv

streamlit run app.py

# ✓ App available at: http://localhost:8501
```

14.6 Step 5 — Verify Setup

1. Open http://localhost:8000/docs — confirm all 16 routes load without error
2. Open http://localhost:8501 — confirm NMK Certification Portal login page loads
3. Login with an @nmkglobalinc.com Cognito account — confirm Admin Dashboard appears
4. Create a test exam (language: python, 5 questions, 5 minutes)
5. Assign to a test candidate email — verify Cognito user created and email received
6. Login as candidate — confirm exam appears in Available Exams

7. Start exam, answer questions, submit — confirm results page with score

15. Key User Workflows

15.1 Admin: Full Exam Lifecycle

#	Actor	Action	System Response
1	**Admin**	Login with @nmkglobalinc.com email	Cognito returns IdToken. /auth/sync sets is_admin=True. Redirect to Admin Dashboard.
2	**Admin**	Fill Create Exam form: title='python', language='python', 10 questions, 30 min	POST /admin/exams: LLM API called in batches. All questions + exam committed atomically.
3	**Admin**	Click Manage Exams tab — confirm exam appears as Active	GET /admin/exams. Exam shown with ▯ status, 10 questions, 30 min.
4	**Admin**	Enter candidate emails in Assign Exams, click Assign	POST /admin/exams/{id}/assign. New users auto-provisioned in Cognito + DB. Emails sent.
5	**Admin**	View Candidate Results tab after candidate completes exam	GET /admin/candidates/results. Shows score, timing, status for all attempts.

15.2 Candidate: First-Time Onboarding

#	Actor	Action	System Response
1	**Candidate**	Receive assignment email with portal URL and temporary password	Email sent by send_exam_assignment_email(send_password=True) via Gmail SMTP.
2	**Candidate**	Login at localhost:8501 with email + temporary password	Cognito validates. IdToken returned. /auth/sync creates candidate DB record.
3	**Candidate**	(Optional) Change password via sidebar Change Password page	POST /auth/change-password-admin verifies old password then sets new permanent password.
4	**Candidate**	Click Start Exam on Available Exams page	POST /exam/{id}/start. CandidateExam created (status=in_progress). Redirected to exam UI.
5	**Candidate**	Answer questions, navigate pages (answers auto-saved)	POST /exam/{id}/bulk-save called on each page navigation via save_dirty_answers().
6	**Candidate**	Click Submit Exam	save_dirty_answers() + POST /exam/{id}/submit. Score computed. Results page shown.

15.3 Session Resume (Browser Refresh)

#	Actor	Action	System Response
1	**Candidate**	Close browser or refresh during exam	CandidateExam remains

#	Actor	Action	System Response
			status=in_progress. All saved answers preserved in DB.
2	Candidate	Re-login with same credentials	Cognito re-issues IdToken. Frontend calls /auth/sync and /auth/me.
3	System	auto_resume_exam_if_needed() called automatically	GET /exam/resume returns full exam state: questions, answers, time_elapsed.
4	Frontend	Restores exam UI with all previous answers pre-filled	Timer re-seeded from (time_allowed_secs - time_elapsed). Candidate continues seamlessly.

16. Complete API Flow Reference (Frontend → Backend → DB)

Every frontend user action mapped: what data is sent, which endpoint handles it, what the backend does to the DB and AWS services, and what is returned to the UI.

16.1 Authentication Flows

Frontend Action	Data Sent	API Endpoint	Response / Side Effects
Login form submitted	Cognito SDK: {USERNAME, PASSWORD}	Cognito initiate_auth()	Returns IdToken + AccessToken. Both stored in session_state.
Post-login backend sync	Bearer IdToken header	POST /auth/sync	Decodes JWT. Upserts User. Sets is_admin for @nmkglobalinc.com. Returns {message}.
Get user info	Bearer IdToken header	GET /auth/me	Returns {email, is_admin, name} from local DB user record.
Forgot password — send code	Cognito SDK: {email}	Cognito forgot_password()	Cognito emails 6-digit reset code to user.
Forgot password — confirm	Cognito SDK: {email, code, new_password}	Cognito confirm_forgot_password()	Password updated. Redirect to login.
Change password	Bearer + {current_password, new_password}	POST /auth/change-password-admin	Verifies old via ADMIN_USER_PASSWORD_AUTH. Sets new permanent password.

16.2 Admin — Exam Creation

Frontend Action	Data Sent	API Endpoint	Response / Side Effects
Create Exam form submitted	Bearer + {title, language, question_count, time_allowed_secs}	POST /admin/exams	LLM called in batches. Exam + Questions committed atomically. Returns ExamOut.

Frontend Action	Data Sent	API Endpoint	Response / Side Effects
Manage Exams tab loads	Bearer token	GET /admin/exams	Returns all exams ordered by created_at DESC.
Deactivate/Activate button	Bearer + path: exam_id	PATCH /admin/exams/{id}/toggle	Flips is_active in DB. Returns {msg, is_active}.
View assignments	Bearer + path: exam_id	GET /admin/exams/{id}/assignments	Returns [{candidate_email, assigned_at}] for this exam.

16.3 Admin — Candidate Assignment

Frontend Action	Data Sent	API Endpoint	Response / Side Effects
Assign Exam button clicked	Bearer + {candidate_emails: [email1, email2, ...]}	POST /admin/exams/{id}/assign	Per email: check DB → if absent: create Cognito user + set permanent password → create DB User → create ExamAssignment → send Gmail email. Idempotent.
View Candidate Results	Bearer token	GET /admin/candidates/results	Returns all CandidateExam records joined with user + exam data.

16.4 Candidate — Exam Taking

Frontend Action	Data Sent	API Endpoint	Response / Side Effects
Exams page loads	Bearer token	GET /exams	Filters ExamAssignment by candidate email + Exam.is_active=True. Returns assigned active exams.
Start Exam button	Bearer + path: exam_id	POST /exam/{id}/start	Creates CandidateExam (in_progress). Links question_ids. Updates assignment.status=started. Returns CandidateExamCreateOut.
Exam page loads	Bearer + path: candidate_exam_id	GET /exam/{id}	Returns questions (without answer_index) + timing. answer_index deliberately excluded.
Page navigation (Next/Prev)	Bearer + {answers:[{question_id, selected_index, time_elapsed}]}	POST /exam/{id}/bulk-save	Saves changed answers to candidate_exams.answers JSON. Updates time_elapsed. flag_modified() forces DB update.
Submit Exam button	Bearer + {final_time_elapsed}	POST /exam/{id}/submit	status=completed, ended_at=now(). compute_score() iterates questions, compares

Frontend Action	Data Sent	API Endpoint	Response / Side Effects
			answer_index. Returns {score, status}.
Results page loads	Bearer + path: candidate_exam_id	GET /exam/{id}/result	Returns score + per-question {text, choices, selected, correct_index, is_correct}. correct_index now revealed.
Login (auto-resume)	Bearer token	GET /exam/resume	If in_progress CandidateExam exists: returns full state including answers dict + time_elapsed for timer re-seed.

17. Email Notification System

All transactional emails are sent via Gmail SMTP using send_exam_assignment_email() in email_utils.py. Connection uses STARTTLS on port 587 with a Gmail App Password for authentication.

17.1 Email Events Reference

Trigger Event	Function	Recipient	Includes Password
New candidate assigned to exam	send_exam_assignment_email(send_password=True)	Candidate	▯ Yes — DEFAULT_PASSWORD from .env
Existing candidate assigned to exam	send_exam_assignment_email(send_password=False)	Candidate	▯ No — 'use your existing password' message

17.2 Email Content Structure

Field	Value
Subject	'NMK Certification Exam Assigned'
From	EMAIL_FROM env variable (sender Gmail address)
To	Candidate's email address
Body — Exam Title	exam_title parameter passed from the assign endpoint
Body — Credentials	Email address + temporary password (if new user) or 'use your existing password'
Body — Portal URL	PORTAL_URL env variable — direct link to login page
Body — Sign-off	'Regards, NMK Certification Team'

17.3 SMTP Connection Code

```
email_utils.py
# email_utils.py — core sending logic
with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
    server.starttls()                          # Upgrade to TLS encryption
    server.login(EMAIL_FROM, EMAIL_PASSWORD)   # App Password auth
    server.send_message(msg)                   # Send EmailMessage object
```

Known Issue: Debug print() statements in email_utils.py expose SMTP_SERVER and EMAIL_FROM to stdout logs. Replace with Python logging module before production deployment.

18. Analytics & Reporting

18.1 Admin Results Dashboard

Endpoint: GET /admin/candidates/results | Auth: Admin only | Page: admin_dashboard() → Candidate Results tab

Metric	Source	Description
Total Attempts	COUNT(CandidateExam records)	All exam attempts matching current filters (status + exam title)
Completed	COUNT WHERE status='completed'	Attempts that were fully submitted and scored
In Progress	COUNT WHERE status='in_progress'	Attempts currently running — candidate has started but not submitted
Avg Score	MEAN(score) for completed	Average percentage score across all completed attempts. Computed in Streamlit via pandas.
Results Table	All CandidateExam joined User + Exam	candidate_name, candidate_email, exam_title, exam_language, status, score, started_at, ended_at
Filter by Status	Client-side pandas filter	Multi-select: completed, in_progress, not_started, timed_out
Filter by Exam	Client-side pandas filter	Multi-select: filter by exam_title values in the results DataFrame

18.2 Score Computation (exam.py)

Field	Source	Description
score	compute_score() → candidate_exams.score	Integer percentage: (correct_answers / total_questions) × 100
Pass threshold	Frontend display logic	≥70% = Pass (green), 50–69% = Borderline (warning), <50% = Fail (error)
Question correctness	selected_index == answer_index	Boolean per question. correct_index revealed in /result endpoint post-submission only.

Field	Source	Description
Unanswered questions	sel is None → skipped	No penalty for unanswered questions — treated as wrong but no negative marking.

19. Known Issues & Recommendations

Priority	Area	Issue	Recommended Fix
Critical	main.py	GET /exams route defined twice with same function name list_all_exams — admin version is shadowed and unreachable at runtime.	Rename to list_admin_exams() and list_candidate_exams() respectively.
High	main.py	CORSMiddleware allow_origins=['*'] — permits requests from any domain in production.	Set allow_origins=['https://portal.nmkglobalinc.com'] for production deployment.
Medium	email_utils.py	print() debug statements expose SMTP_SERVER and EMAIL_FROM to stdout logs.	Replace all print() calls with Python logging module at appropriate log levels.
Medium	main.py	Auto-provisioned candidates get id=email string instead of their Cognito sub UUID, causing mismatch on first real login.	Call admin_get_user() after admin_create_user() to get actual sub UUID for User.id.
Medium	main.py	All candidates share DEFAULT_PASSWORD with no forced change on first login.	Use Permanent=False in admin_set_user_password() and handle NEW_PASSWORD_REQUIRED challenge in frontend.
Low	app.py	No JWT token refresh logic — IdToken expires after 60 minutes causing silent 401 errors.	Implement token refresh using the Cognito RefreshToken stored in session state on 401 responses.
Low	app.py	Exam countdown timer is client-side JavaScript only — can be manipulated by the candidate.	Server-side validation: reject submission if (ended_at - started_at) > (time_allowed_secs + grace_period).
Low	exam.py	compute_score() contains extensive print() debug statements logging all question/answer comparisons.	Remove or gate behind a DEBUG flag before production.

20. Glossary

Term	Definition
JWT	JSON Web Token — a signed, URL-safe token encoding user identity claims. Format: header.payload.signature (base64url encoded).
JWKS	JSON Web Key Set — AWS Cognito's published RSA public keys used to verify JWT signatures without contacting Cognito on every request.
RS256	RSA + SHA-256 — asymmetric JWT signing. Cognito signs with a private key; backend verifies using the public key from JWKS.

Term	Definition
IdToken	The specific Cognito JWT token containing user identity claims (email, sub, name). Used as Bearer token throughout this system.
sub	Subject — unique Cognito UUID assigned to each user. Immutable. Should be used as the User.id primary key in the local DB.
ORM	Object-Relational Mapper — SQLAlchemy maps Python classes to PostgreSQL tables, abstracting raw SQL queries.
STARTTLS	Protocol upgrade on SMTP port 587: starts as a plain TCP connection then upgrades to TLS before any credentials are transmitted.
boto3	Official AWS Python SDK. Used here for Cognito admin operations: admin_create_user() and admin_set_user_password().
RBAC	Role-Based Access Control — access decisions based on user role. Enforced via is_admin DB flag on all /admin/* endpoints.
flag_modified()	SQLAlchemy utility to explicitly mark a mutable JSON column as changed — required for dict mutations to be detected and persisted in UPDATE.
CandidateExam	Database record for one candidate's attempt at one exam. Stores ordered question IDs, all answers, timing data, status, and final score.
LLM	Large Language Model — external AI service generating MCQ exam questions on demand by programming language and quantity.
MCQ	Multiple Choice Question — a question format with exactly one correct answer selected from a list of typically 4 options.
gen_id()	Helper function: str(uuid.uuid4()). Used as the default= for all UUID primary key columns in ORM models.

END OF DOCUMENT

NMK Certification Portal — Complete Technical Documentation

Author & Maintainer: V. Prathyusha | Developers: Sushmitha, Sai Kiran, Sohail, Gopi Kiran